TECHNICAL REPORT

Movie Explorer

Flutter Riverpod State Management Application

App Name	Movie Explorer
Framework	Flutter 3.x (Dart 3.11.5)
State Management	Flutter Riverpod 2.6.1 + Hooks Riverpod 2.6.1
External API	Watchmode API (v1)
Version	1.0.0+1
Report Date	09 June 2026

Table of Contents

1. Project Overview

Movie Explorer is a Flutter mobile application designed and built as a practical learning exercise in Riverpod state management. The application demonstrates the full Riverpod provider ecosystem within a real-world movie discovery context, integrating with the Watchmode REST API to deliver live search results, detailed movie information, and streaming availability data.

The project was developed as part of a structured Riverpod learning curriculum, covering Provider, StateProvider, NotifierProvider, FutureProvider, and FutureProvider.family — the five core provider types used in modern Flutter applications. It also includes flutter_hooks and hooks_riverpod as optional companion packages.

1.1 Application Purpose

The app serves two purposes simultaneously: a fully functional movie discovery tool for end users, and a clean reference codebase for developers learning Riverpod state management patterns. Every provider in the app is intentionally scoped and typed to demonstrate best practices.

1.2 Learning Objectives Covered

- Understand how Riverpod improves upon the legacy Provider package
- Implement compile-time safety and testable providers without BuildContext dependency
- Use ProviderScope, ConsumerWidget, Consumer, ref.watch, and ref.read correctly
- Integrate FutureProvider for asynchronous REST API calls
- Manage complex list state using NotifierProvider
- Apply StateProvider for simple primitive state (search query, theme toggle)
- Understand the lifecycle and reactivity model of Riverpod providers

1.3 Key Features

- Live movie and series search via the Watchmode autocomplete API
- Detailed movie view: plot, rating, runtime, genres, and streaming sources
- Persistent in-session Favourites list (heart / remove toggle)
- Persistent in-session Watchlist (bookmark / remove toggle)
- Dark/Light theme toggle managed by a dedicated NotifierProvider
- Full-bleed hero poster layout on the Details screen
- Custom cinematic design system: deep slate palette with electric-teal accents
- Portrait-locked, Material 3, custom fonts (Bebas Neue + DM Sans)

2. Application Architecture

Movie Explorer follows a layered architecture pattern with a clear separation of concerns between data, state, and UI layers. Riverpod providers act as the glue between the service layer (API calls) and the presentation layer (screens and widgets).

2.1 Project Structure

The lib/ directory is organised into the following packages:

Directory / File	Responsibility
lib/main.dart	App entry point; ProviderScope root; MaterialApp with theme wiring
lib/core/theme/	Centralised AppTheme (dark + light) and AppColors palette constants
lib/models/	Dart data classes (Movie) with factory JSON constructors
lib/services/	MovieService — raw HTTP calls to Watchmode API; no Riverpod coupling
lib/providers/	All Riverpod provider declarations: movie, favourite, watchlist, theme
lib/screens/	Top-level UI screens: Home, Search, Details, Favourite, Watchlist, Settings
lib/widgets/	Reusable UI components: MovieCard, MovieSearchBar, EmptyState
lib/routes/	AppRoutes — named route constants and generateRoute factory

2.2 Architecture Layers

The application uses a three-tier architecture:

- Data Layer — MovieService performs raw HTTP GET requests using the http package. It is injected into Riverpod via movieServiceProvider (a Provider<MovieService>), enabling easy mocking in tests.
- State Layer — Riverpod providers bridge the data layer and UI. FutureProviders manage async API calls; NotifierProviders manage mutable lists; StateProvider holds the search query string.
- Presentation Layer — Screens extend ConsumerWidget and access providers through WidgetRef (ref.watch / ref.read). Screens never call HTTP directly.

2.3 Navigation

Navigation is implemented using a custom AppRoutes class with a generateRoute factory method. Named route strings (/, /search, /favorites, /watchlist) are declared as static constants. The HomeScreen uses an IndexedStack with a NavigationBar to switch between tabs without rebuilding child screens, preserving scroll positions across tab changes.

Push navigation to the Details screen is handled with MaterialPageRoute inside the SearchScreen, passing the selected Movie object as a constructor argument.

3. Riverpod Provider Analysis

Movie Explorer uses five distinct provider types, each chosen for a specific responsibility. This section documents each provider, its type rationale, how it is consumed, and key implementation details.

3.1 Provider — movieServiceProvider

File: lib/providers/movie_provider.dart
Type: Provider<MovieService>
Purpose: Dependency injection for the HTTP service layer

A read-only Provider<T> is ideal for singleton dependencies that never change after construction. movieServiceProvider creates a single MovieService instance and makes it accessible to other providers (not directly to widgets), enabling the service layer to be replaced or mocked in tests without modifying any screen code.

```
final movieServiceProvider = Provider<MovieService>((ref) {
  return MovieService();
});
```

3.2 StateProvider — searchQueryProvider

File: lib/providers/movie_provider.dart
Type: StateProvider<String>
Purpose: Stores the live search query string as the user types

StateProvider is the correct choice for simple, primitive state that widgets write to and other providers read from. When the user types in the search field, the onChanged callback writes directly to the provider state. The downstream searchMoviesProvider automatically re-evaluates whenever the query changes, creating a fully reactive search pipeline with no manual state management.

```
final searchQueryProvider = StateProvider<String>((ref) => '');
```

Consumed in SearchScreen via:

```
ref.read(searchQueryProvider.notifier).state = value;
```

3.3 FutureProvider — searchMoviesProvider

File: lib/providers/movie_provider.dart
Type: FutureProvider<List<Movie>>
Purpose: Async movie search results from Watchmode autocomplete API

FutureProvider automatically wraps an async operation in an AsyncValue<T>, exposing three states: loading, data, and error. The provider watches searchQueryProvider — when the query updates, Riverpod cancels the previous Future and fires a new one, preventing stale results. An empty query guard returns an empty list immediately without hitting the API.

```
final searchMoviesProvider = FutureProvider<List<Movie>>((ref) async {
  final query = ref.watch(searchQueryProvider);
  if (query.isEmpty) return [];
  final service = ref.watch(movieServiceProvider);
  return service.searchMovies(query);
});
```

Consumed in SearchScreen with the .when() pattern:

```
movies.when(
  data: (list) => ListView.builder(...),
  loading: () => CircularProgressIndicator(),
  error: (e, _) => Text('Something went wrong'),
)
```

3.4 FutureProvider.family — movieDetailsProvider & movieSourcesProvider

File: lib/providers/movie_provider.dart
Type: FutureProvider.family<Map<String,dynamic>, int>
Purpose: Per-movie detail and streaming sources fetched by movie ID

FutureProvider.family creates a parameterised provider — one provider family can generate a separate, cached provider instance per unique argument. Both movieDetailsProvider and movieSourcesProvider are keyed by the integer movie ID, ensuring each movie's details are only fetched once per app session. The Details screen passes the movie's ID to access the correct instance.

```
final movieDetailsProvider =
    FutureProvider.family<Map<String, dynamic>, int>((ref, movieId) async {
  final service = ref.watch(movieServiceProvider);
  return service.getMovieDetails(movieId);
});
```

Consumed in DetailsScreen as:

```
final detailsAsync = ref.watch(movieDetailsProvider(movie.id));
```

3.5 NotifierProvider — favoriteProvider

File: lib/providers/favorite_provider.dart
Type: NotifierProvider<FavoriteNotifier, List<Movie>>
Purpose: In-session list of user-favourited movies with toggle logic

NotifierProvider is the modern replacement for StateNotifierProvider (deprecated in Riverpod 2.x). FavoriteNotifier extends Notifier<List<Movie>> and centralises all mutation logic: toggleFavorite checks for duplicates by movie ID before adding, and removes by filter rather than index for safety. The state is a new list reference each time it is mutated, ensuring widgets re-render correctly.

```
class FavoriteNotifier extends Notifier<List<Movie>> {
  @override
  List<Movie> build() => [];

  void toggleFavorite(Movie movie) {
    final exists = state.any((item) => item.id == movie.id);
    if (exists) {
      state = state.where((item) => item.id != movie.id).toList();
    } else {
      state = [...state, movie];
    }
  }
}
```

Consumed in two ways: ref.watch (read-only display) and ref.read (mutation triggers):

```
final favorites = ref.watch(favoriteProvider);           // reactive list
ref.read(favoriteProvider.notifier).toggleFavorite(m);  // mutation
```

3.6 NotifierProvider — watchlistProvider

File: lib/providers/watchlist_provider.dart
Type: NotifierProvider<WatchlistNotifier, List<Movie>>
Purpose: In-session watchlist (queue of movies to watch later)

WatchlistNotifier mirrors the FavoriteNotifier pattern, demonstrating how the same provider type can manage two independent state trees for different features. Both providers share an identical public API (toggleX, isInX) which simplifies the Details screen's action button wiring.

3.7 NotifierProvider — themeProvider

File: lib/providers/theme_provider.dart
Type: NotifierProvider<ThemeNotifier, bool>
Purpose: Dark/light theme toggle — bool state (true = dark)

ThemeNotifier holds a single boolean and exposes a toggleTheme() method. The root MovieApp widget (a ConsumerWidget) watches this provider and passes isDark ? ThemeMode.dark : ThemeMode.light to MaterialApp, making the theme reactive to the Settings screen toggle with zero additional wiring.

```
class ThemeNotifier extends Notifier<bool> {
  @override bool build() => false;
  void toggleTheme() => state = !state;
}
```

4. Screens and User Interface

4.1 HomeScreen

HomeScreen is a standard StatefulWidget (not ConsumerWidget) because it only manages local navigation state (_currentIndex). It wraps four tab screens in an IndexedStack, ensuring each screen's state (e.g. scroll position, loaded data) survives tab switches. The NavigationBar uses Material 3's NavigationDestination with rounded icons and custom teal indicator colours.

4.2 SearchScreen

SearchScreen extends ConsumerWidget and demonstrates the full FutureProvider consumption pattern. The MovieSearchBar widget calls onChanged to update searchQueryProvider. Because searchMoviesProvider watches searchQueryProvider, every keystroke triggers a re-evaluation. The .when() idiom renders a loading spinner, an error message, a prompt empty state (no query), or the movie list depending on the AsyncValue state.

4.3 DetailsScreen

The Details screen is the most complex screen in the application. It uses a layered Stack layout: a full-bleed poster image occupies the top 55% of the screen, a gradient overlays the top edge for back-button contrast, and a scrollable bottom panel with rounded top corners slides up over the poster. Additional details (plot, rating, runtime, genres) are fetched via movieDetailsProvider.family and rendered using .when(). Two AnimatedContainer action buttons (Favourite and Watchlist) animate between active/inactive states using the respective NotifierProviders.

4.4 FavoriteScreen

FavoriteScreen uses ref.watch(favoriteProvider) to reactively display the current favourites list. When the list is empty, the EmptyState widget renders a teal-ringed icon with instructional copy. Non-empty lists are rendered via ListView.builder using MovieCard with an onDelete callback that calls toggleFavorite, immediately removing the item from the list and triggering a UI rebuild.

4.5 WatchlistScreen

WatchlistScreen mirrors FavoriteScreen with watchlistProvider and gold accent colours (bookmarkGold). A count badge appears in the header when the list is non-empty, rendered using a coloured container with semi-transparent background — the same pattern as the Favourites badge.

4.6 SettingsScreen

SettingsScreen demonstrates reading a NotifierProvider for UI state (isDark) and writing to it via a Switch widget. The settings are grouped in a styled container tile with rounded corners and a border. The screen also shows an About section with the app version (1.0.0), illustrating a static settings tile pattern.

5. Service and Data Layer

5.1 MovieService

MovieService is a plain Dart class that handles all HTTP communication with the Watchmode API. It is not a Riverpod provider itself — it is injected via movieServiceProvider (a Provider<MovieService>), keeping the service class pure and independently testable. The class exposes three async methods:

- searchMovies(String query) — Calls the /v1/autocomplete-search/ endpoint with search_type=2 (titles only). Returns a List<Movie> parsed from the results array.
- getMovieDetails(int movieId) — Calls /v1/title/{movieId}/details/ and returns the raw JSON map. Fields are extracted in the Details screen.
- getSources(int movieId) — Calls /v1/title/{movieId}/sources/ and returns a list of streaming source objects.

All three methods use the http.get() function with an X-API-Key header. On non-200 responses, they throw an Exception, which Riverpod's FutureProvider automatically captures into the error state of AsyncValue, displaying the error gracefully in the UI.

5.2 Movie Model

The Movie class is a simple immutable Dart data class with six fields: id (int), title, type, year, poster, and description. The factory Movie.fromJson() constructor maps Watchmode API response fields to the model, with null-safe fallbacks for optional fields (e.g. description falls back from plot_overview to description to a default string).

5.3 API Integration Details

Endpoint	Provider	Response
/autocomplete-search/	searchMoviesProvider	Array of title objects with id, name, type, year, image_url
/title/{id}/details/	movieDetailsProvider	Single title object with plot_overview, user_rating, runtime_minutes, genre_names
/title/{id}/sources/	movieSourcesProvider	Array of streaming source objects (declared, not yet rendered in UI)

6. Design System

6.1 Colour Palette

The app uses a "cinematic dark luxury" palette defined in AppColors (lib/core/theme/app_theme.dart). All colour constants are used throughout the codebase via the AppColors namespace, ensuring a single source of truth.

Constant	Hex Value	Usage
AppColors.bg	#0F1923	Scaffold background, details panel background
AppColors.surface	#1C2B3A	Card surfaces, navigation bar background
AppColors.surfaceHigh	#243447	Elevated cards, search field fill
AppColors.teal	#00C9A7	Primary accent: accent strips, active icons, borders
AppColors.heartRed	#FF6B81	Favourites icon, Favourites screen header accent
AppColors.bookmarkGold	#FFD166	Watchlist icon, Watchlist screen header accent
AppColors.textPrimary	#F0F4F8	Headings and primary text
AppColors.textSecondary	#8FA8BE	Body copy, card metadata
AppColors.textMuted	#4A6278	Placeholder text, inactive icons, labels

6.2 Typography

The app uses two custom font families bundled in assets/fonts/:

- Bebas Neue — Display-weight face used for displayLarge/displayMedium text styles. Its condensed, all-caps letterforms reinforce the cinematic poster aesthetic.
- DM Sans — A geometric sans-serif used for all UI text (headings, body, labels). Its wide character set and high x-height ensure readability at small sizes on mobile screens.

The text theme is defined once in AppTheme._textTheme and covers all 13 Material 3 text roles from displayLarge (54pt) down to labelSmall (10pt), with letter-spacing and line-height tuned for each role.

6.3 Component Patterns

- MovieCard — A 90dp-tall row card with a left teal accent strip, a poster thumbnail with right-fade gradient, title and type/year chips, and a conditional trailing widget (arrow for navigation, delete icon for lists).
- EmptyState — A centred column with a teal-ringed circular icon container, title text, and subtitle. Reused across Search, Favourites, and Watchlist empty states with different icon/copy.

- _MetaChip / _Chip — Small rounded containers used to display type and year metadata on cards and the details screen. Teal chips use tealDim fill; neutral chips use surfaceHigh fill.
- _ActionButton — An AnimatedContainer that smoothly transitions border colour and background opacity based on isActive state, used for the Favourite and Watchlist toggles on the Details screen.

7. Tools and Technologies

Package / Tool	Version	Purpose
Flutter SDK	3.x	Cross-platform UI framework (Android, iOS, Web, Desktop)
Dart SDK	^3.11.5	Programming language; null safety, strong typing
flutter_riverpod	^2.6.1	Core Riverpod package; Provider, StateProvider, FutureProvider, NotifierProvider
hooks_riverpod	^2.6.1	Combines flutter_hooks and riverpod; HookConsumerWidget support
flutter_hooks	^0.21.2	React-style hooks (useState, useEffect, useTextEditingController) for Flutter
http	^1.5.0	Lightweight HTTP client used in MovieService for REST API calls
cached_network_image	^3.4.1	Network image loading with in-memory and disk caching for poster images
shared_preferences	^2.5.3	Key-value persistence (installed, not yet used — planned for favourites persistence)
go_router	^16.0.0	Declarative routing (installed; current routing uses manual AppRoutes for simplicity)
Watchmode API	v1	External REST API for movie/series search, details, and streaming sources
Bebas Neue Font	—	Display typeface for cinematic poster-style headings
DM Sans Font	—	UI sans-serif for all body and label text
Flutter Lints	^6.0.0	Recommended lint rules for code quality and consistency

8. State Management Comparison

A central learning objective of this project is understanding when and why to use Riverpod over older Flutter state management approaches. This section compares three patterns used across Flutter projects.

8.1 setState

setState is Flutter's most basic state management mechanism. It triggers a rebuild of the widget that called it and all its descendants.

- Appropriate for: purely local, ephemeral state (input focus, animation phase, toggle inside a single widget).
- Limitations: state cannot be shared across widgets; leads to prop-drilling (passing state and callbacks down the tree); very difficult to test because state lives inside StatefulWidget instances.
- Example in this project: HomeScreen uses setState for _currentIndex (bottom navigation tab), which is genuinely local.

8.2 Provider (Legacy)

The Provider package (by Remi Rousselet, pre-Riverpod) solved state sharing by placing state objects in the widget tree via InheritedWidget. Widgets read state with context.read() and context.watch().

- Advantages over setState: state is shareable across the tree; separates state logic from UI.
- Limitations: depends on BuildContext — providers cannot be accessed outside of the widget tree; difficult to test in isolation; no compile-time safety (typos in provider types are runtime errors); accidental overrides in nested scopes are a known footgun.

8.3 Riverpod

Riverpod is a complete rewrite of Provider that removes the BuildContext dependency entirely. Providers are top-level global constants, resolved at runtime through ProviderScope.

Feature	Provider (Legacy)	Riverpod
BuildContext required?	Yes	No
Compile-time safety	No (runtime errors)	Yes
Testing without Flutter	Difficult	Easy (ProviderContainer)
Async state (AsyncValue)	Manual (FutureBuilder)	Built-in
Family (parameterised)	Not supported	Yes (.family modifier)
Accidental scope overrides	Possible	Prevented by design
Deprecated provider types	None (but package is legacy)	StateNotifier (use Notifier)

8.4 When to Use Each

- Use setState for truly local UI state that no other widget needs to know about (e.g., a toggle inside a single card, animation progress).
- Use Riverpod StateProvider for simple shared primitive state (strings, booleans, enums) that is read by multiple widgets or other providers.
- Use Riverpod NotifierProvider for complex shared state with multiple mutation methods (lists, objects with business logic).
- Use Riverpod FutureProvider for async data — it replaces FutureBuilder with automatic loading/error/data lifecycle management.
- Use Riverpod FutureProvider.family for async data that is keyed by a parameter (e.g., details per movie ID, user data per user ID).
- Avoid legacy Provider in new projects — it has no new features planned and Riverpod is its designated successor.

9. Key Implementation Patterns

9.1 ProviderScope at the Root

All Riverpod providers require a ProviderScope ancestor. In Movie Explorer, ProviderScope wraps MovieApp directly in main.dart's runApp() call, ensuring the provider container is available throughout the entire widget tree. No provider can be accessed outside of a ProviderScope.

9.2 ConsumerWidget vs Consumer

ConsumerWidget is the primary way to access providers in Movie Explorer. It provides a WidgetRef ref parameter to the build() method, used with ref.watch() for reactive reads and ref.read() for one-shot reads (e.g., inside callbacks and onTap handlers). Consumer is an inline alternative that wraps a sub-tree, used when only a small part of a large widget needs to rebuild on provider changes.

- ref.watch(provider) — subscribes to changes; rebuilds the widget when the provider value changes. Used in build().
- ref.read(provider) — reads once without subscribing. Used in callbacks (onTap, onChanged) to avoid unnecessary rebuilds.
- ref.read(provider.notifier) — accesses the Notifier object to call mutation methods on a NotifierProvider.

9.3 Reactive Search Pipeline

The search feature demonstrates Riverpod's reactive chain: user types → searchQueryProvider updates → searchMoviesProvider (which watches it) automatically re-runs → SearchScreen (which watches searchMoviesProvider) rebuilds. This three-step chain is driven entirely by Riverpod's dependency graph, with no manual event handling or StreamControllers required.

9.4 AsyncValue.when() Pattern

Every FutureProvider is consumed with the .when() pattern, which forces the developer to handle all three possible states. This pattern eliminates unchecked null access, missing loading spinners, and unhandled error states — common bugs in naive FutureBuilder implementations.

9.5 Immutable State Updates

Both FavoriteNotifier and WatchlistNotifier update state by creating new list instances rather than mutating the existing list. This is required for Riverpod to detect the change and notify listeners. The spread operator pattern ([...state, movie]) and .where().toList() pattern are the idiomatic approaches.

10. Output and Deliverables

10.1 Application Deliverables

- Fully functional Flutter application targeting Android and iOS
- Live movie and series search via the Watchmode REST API
- In-session Favourites and Watchlist management with toggle logic
- Dark/Light theme toggle persisting across screen navigations in-session
- Full-bleed hero poster UI on the Details screen with overlaid metadata
- Custom cinematic design system with BebasNeue and DM Sans fonts

10.2 Riverpod Provider Types Demonstrated

Provider Type	Instance	Use Case
Provider<T>	movieServiceProvider	Singleton service injection
StateProvider<T>	searchQueryProvider	Primitive reactive state (search string)
FutureProvider<T>	searchMoviesProvider	Async API result list
FutureProvider.family<T,A>	movieDetailsProvider, movieSourcesProvider	Parameterised async data per movie ID
NotifierProvider<N,T>	favoriteProvider, watchlistProvider, themeProvider	Mutable state with business logic methods

10.3 State Management Examples

- Reactive search: StateProvider + FutureProvider dependency chain
- List management: NotifierProvider with immutable state updates
- Theme switching: NotifierProvider<bool> wired to MaterialApp themeMode
- Per-ID async data: FutureProvider.family keyed by movie ID
- Service injection: Provider<T> for testable HTTP client
- Local UI state: setState in HomeScreen for tab index

10.4 Supporting Documentation

- This Technical Report — detailed architectural and code-level documentation
- README.md — setup instructions, dependency list, and usage guide
- Inline code comments — providers, notifiers, and key widget patterns are annotated

11. Known Limitations and Future Improvements

11.1 Current Limitations

- Favourites and Watchlist are in-session only — no persistence. The shared_preferences package is installed but not yet wired. On app restart, both lists reset to empty.
- go_router is installed but unused — the current AppRoutes implementation uses the legacy generateRoute factory. Migrating to go_router would enable deep linking and URL-based navigation.
- movieSourcesProvider is fetched but the streaming sources data is not yet rendered in the Details screen UI.
- The search fires on every keystroke — a debounce (e.g., 300ms) would reduce unnecessary API calls and improve performance on slow connections.
- API key is hardcoded in MovieService. Production apps must use environment variables or a secrets file excluded from version control.

11.2 Recommended Improvements

- Persist Favourites and Watchlist using shared_preferences via AsyncNotifier so the state survives app restarts.
- Add a debounce to the search input using flutter_hooks (useEffect + useState) to throttle API calls.
- Migrate routing to go_router for named routes, deep linking, and back-button handling on web/Android.
- Add unit tests for FavoriteNotifier and WatchlistNotifier using ProviderContainer (no Flutter widget tree required).
- Render the streaming sources list on the Details screen from movieSourcesProvider.
- Introduce error boundaries: ProviderObserver can log provider errors globally for debugging.

12. Conclusion

Movie Explorer successfully demonstrates the Riverpod 2.x provider ecosystem within a production-quality Flutter application. All five major provider types are used with clear rationale: Provider for dependency injection, StateProvider for simple primitive state, FutureProvider for async API integration, FutureProvider.family for parameterised async data, and NotifierProvider for complex mutable state with encapsulated business logic.

The reactive search pipeline — from user keystroke through StateProvider to FutureProvider to UI rebuild — is a concise, real-world demonstration of Riverpod's composability. The NotifierProvider pattern for Favourites and Watchlist shows how complex list state with toggle logic can be managed cleanly without any setState or StreamController plumbing.

Compared to legacy Provider and setState, Riverpod offers compile-time safety, BuildContext independence, and first-class support for async state that materially reduces the boilerplate and error surface in production Flutter apps. This project serves as both a working reference implementation and a foundation for further Riverpod exploration, including persistence, testing, and advanced patterns such as StreamProvider and Ref invalidation.

End of Technical Report — Movie Explorer / Flutter Riverpod State Management Application